Q-MTLS Benchmark — Student Task

Read this once end-to-end. If you hit something not covered here, ping me before improvising.

Your mission (one sentence)

Collect ≥300 distinct news article URLs per sub-topic, for the four C9 sub-topics, using the GDELT 2.0 DOC API, and write them out as `article_urls.jsonl` in the schema we already use.

Nothing else. Don't touch the timelines, the datasheet, or anything in `c1_*` through `c8_*` and `c10_*` through `c12_*`.

The four sub-topics

Sub-topic id	Event	Approximate window
`c9_a_turkey_syria_eq`	Turkey-Syria earthquakes	2023-02-01 → 2023-04-30
`c9_b_morocco_eq`	Morocco earthquake	2023-09-01 → 2023-12-15
`c9_c_libya_floods`	Libya / Derna floods (Storm Daniel)	2023-09-01 → 2023-11-30
`c9_d_maui_fires`	Hawaii / Maui / Lahaina wildfires	2023-08-01 → 2023-10-31

The exact keywords are already written for you — see `c9_disasters_2023/subtopics/<sub-topic>/keywords.json`. Use those words verbatim in your queries.

Setup (15 minutes)

1. Install Python 3.10+ if you don't have it.
2. `pip install gdeltdoc requests tldextract`
3. Open `harvest_gdelt.py` (in this same folder). It is a starter script — read it once, then run it.
4. Test that GDELT works:

   ```
   python harvest_gdelt.py --subtopic c9_d_maui_fires --limit 50 --dry-run
   ```

   You should see ~50 article URLs print to the screen with their outlet and date. If you see an error like `403` or `connection refused`, GDELT is throttling you — wait 5 minutes and retry.

The task (most of your time)

For each of the four sub-topics, run:

```
python harvest_gdelt.py --subtopic c9_a_turkey_syria_eq
python harvest_gdelt.py --subtopic c9_b_morocco_eq
python harvest_gdelt.py --subtopic c9_c_libya_floods
python harvest_gdelt.py --subtopic c9_d_maui_fires
```

Each run will:

- Query GDELT with the keywords from `keywords.json`, restricted to the date window above.
- Filter out anything in `aggregator_blocklist.json` (Yahoo, MSN, social media, etc.).
- Deduplicate by URL.
- Save to `c9_disasters_2023/subtopics/<sub-topic>/article_urls.jsonl`.
- Print a summary: number of URLs, top 10 outlets, share of top outlet.

Acceptance criteria for each sub-topic:

1. ≥300 distinct URLs (target: 500+).
2. The single largest outlet is ≤40% of the total. If it's higher, broaden the query (try a synonym from `keywords.json`'s `entity_keywords` list) and rerun.
3. Every URL has a `pub_date` (GDELT gives you this for free; don't drop the field).
4. Each `first_paragraph_preview` is ≤200 characters. The starter script already enforces this — don't relax it.
5. No URLs from these domains: `topix.com`, `msn.com`, `yahoo.com`, `dailymail.co.uk`, `rt.com`, `sputnikglobe.com`, plus any social media (already in `aggregator_blocklist.json`).

Verify your work (10 minutes)

Run the validator:

```
cd benchmark
python validate_benchmark.py --cluster c9_disasters_2023
```

You'll get a list of PASS / WARN / FAIL lines. The four `articles.<sid>.count` checks should switch from FAIL to PASS once you cross 300 URLs each. The two `entanglement.*` checks will still fail — that's not your problem, ignore them.

If `articles.<sid>.outlet_diversity` fails (one outlet > 40%), see acceptance criterion #2 above.

What to send back

Just three things:

1. The four updated `article_urls.jsonl` files (or the whole `c9_disasters_2023/` folder zipped).
2. The validator output, copy-pasted into a text file `validation_log.txt`.
3. A short note (3-5 sentences) on anything weird you ran into — outlets that returned junk, queries that returned <50 results, etc.

Don't email me drafts. One clean handover at the end.

Things you must NOT do

- Don't write or edit any `timelines.jsonl`. Those are gold and already done.
- Don't touch `datasheet.md`, `cluster_report.md`, `entanglement_metrics.json`, `coverage_report.json`, or `cluster_meta.json`.
- Don't run scrapers against publisher sites (no `requests` to `nytimes.com/article/...`). GDELT is the **only** source for this task. Site-direct scraping has copyright and TOS implications I don't want you handling solo.
- Don't paste full article text into the JSONL. Only the schema in the starter script — URL, title (≤280 chars from GDELT), outlet, pub_date, first_paragraph_preview (≤200 chars, also from GDELT's seendate / social image alt — never from scraped HTML).
- Don't change the validator thresholds in `validate_benchmark.py`.

If you finish early

Don't start C10 or any other cluster. Instead, do this:

- Run a **duplicate-URL audit**: for each `article_urls.jsonl`, group by the URL with query strings stripped. Report any near-duplicates (same outlet + same path differing only in tracking parameters). I'll merge them.
- Try the same harvest for `c9_a_turkey_syria_eq` once more, but with the GDELT `sourcecountry:syr` filter on (Syrian-perspective coverage) — that often pulls in Reuters/AFP wires that are hidden behind English-search defaults. Report the count separately, don't merge.

Schema reminder (the only schema you will produce)

Each line of `article_urls.jsonl` must look like this:

```
{"id": "c9_a_00001", "url": "https://...", "title": "...",
 "outlet": "Al Jazeera", "pub_date": "2023-02-08",
 "first_paragraph_preview": "Short ≤200-char neutral sentence in your own words."}
```

`id` is `c9_<letter>_NNNNN` (5-digit, zero-padded). The starter script does this for you.